

Supervised Learning

Naive Bayes

The classification setting

- The response variable Y is **qualitative**. Examples:
 - ▶ eye color $\in \{\text{blue}, \text{green}, \text{brown}\} := \mathcal{C}$;
 - ▶ email $\in \{\text{spam}, \text{nospam}\} := \mathcal{C}$.
- **Number of Classes:** K .
- **Response** $Y \in \{1, \dots, K\}$.
- We are interested in estimating the **posterior probabilities** $p_k(\mathbf{x})$,

$$p_k(\mathbf{x}) := \Pr(Y = k | \mathbf{X} = \mathbf{x}), \quad k = \overline{1, K}.$$

Bayes' formula

Thomas Bayes published this result in 1763 (we can think about this formula as a way of inverting conditional probabilities),

$$Pr(Y = k | \mathbf{X} = \mathbf{x}) = \frac{Pr(\mathbf{X} = \mathbf{x} | Y = k) Pr(Y = k)}{Pr(\mathbf{X} = \mathbf{x})}.$$

or

$$Pr(Y = k | \mathbf{X} = \mathbf{x}) = \frac{\pi_k f_k(\mathbf{x})}{\sum_{l=1}^K \pi_l f_l(\mathbf{x})}.$$

- $\pi_k = Pr(Y = k)$ is the **prior** (marginal) probability for class k . The prior probabilities π_k can be estimated by $\hat{\pi}_k$, where $\hat{\pi}_k$ can be taken as the proportion of training observations belonging to class k .
- $f_k(\mathbf{x})$ is the **density** for $\mathbf{X}$ in class k .

The naive Bayes classifier

Estimating the densities $f_k(\mathbf{x})$, $\mathbf{x} \in \mathbb{R}^p$ may be challenging in general. The naive Bayes classifier makes the following assumption:

Within the k^{th} class the predictors $X_1, \dots, X_p$ are independent.

This assumption can be written as:

$$f_k(\mathbf{x}) = f_{k1}(x_1) \times \dots \times f_{kp}(x_p), \quad k = \overline{1, K},$$

where $f_{kj}(x_j)$ is the density function of the j^{th} predictor among observations in the k^{th} class. It follows that

$$Pr(Y = k | \mathbf{X} = \mathbf{x}) = \frac{\pi_k \times f_{k1}(x_1) \times \dots \times f_{kp}(x_p)}{\sum_{l=1}^K \pi_l \times f_{l1}(x_1) \times \dots \times f_{lp}(x_p)}$$

The naive Bayes classifier

Estimating the densities $f_{kj}(x_j)$:

- ▶ X_j is **quantitative**.
 - ▶ (**parametric estimate**) $X_j|Y = k \sim N(\mu_{jk}, \sigma_{jk}^2)$;
 - ▶ (**non-parametric estimate**) To estimate $f_{kj}(x_j)$:
 - ▶ Make a histogram of the j^{th} predictor within each class;
 - ▶ Estimate $f_{kj}(x_j)$ as the fraction of the training observations in the k^{th} class that belong to the same histogram bin as x_j .
- ▶ X_j is **qualitative**. In this case, $f_{kj}(x_j)$ can be approximated by the proportion of training observations for the j^{th} predictor corresponding to class k . **Example.** Let $X_j \in \{1, 2, 3\}$ and suppose that X_j takes on values 1, 2 and 3 in 34, 52, 14 of the observations respectively. Then $\hat{f}_{kj}(x_j) \approx f_{kj}(x_j)$, where

$$\hat{f}_{kj}(x_j) = \begin{cases} 0.34 & \text{if } x_j = 1 \\ 0.52 & \text{if } x_j = 2 \\ 0.14 & \text{if } x_j = 3 \end{cases}$$

Pros and Cons

Pros.

- Works well for many feature;
- Fairly fast to calculate;
- Handles rare events and missing data well.

Cons.

- Needs large amounts of training data;
- Continuous data must be processed to be used;
- Assumes features are independent and equally important.

Naive Bayes in Python

```
In [1]: # Import packages and functions
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
```

```
In [2]: # Read in the data and view the first five instances.
# File does not include column headers so they are provided via names.
messages = pd.read_table('SMSSpamCollection.csv', names=['Class', 'Message'])
messages.head()
```

Out[2]:

	Class	Message
0	ham	Go until jurong point, crazy.. Available only ...
1	ham	Ok lar... Joking wif u oni...
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...
3	ham	U dun say so early hor... U c already then say...
4	ham	Nah I don't think he goes to usf, he lives aro...

Naive Bayes in Python

```
In [3]: # Split into testing and training sets
X_train, X_test, Y_train, Y_test = train_test_split(
    messages['Message'], messages['Class'], random_state=123
)
```

The following cell uses `sklearn CountVectorizer`, which counts the words that appear in the messages and stores them in a Python dictionary using these counts.

You can read more about `CountVectorizer` at

https://scikit-learn.org/1.5/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

```
In [4]: # Count the words that appear in the messages
vectorizer = CountVectorizer(ngram_range=(1, 1))
vectorizer.fit(X_train)
# Uncomment the line below to see the words.
#vectorizer.vocabulary_
```

```
Out[4]: ▼ CountVectorizer
CountVectorizer()
```


Naive Bayes in Python

```
In [5]: # Count the words in the training set and store in a matrix  
X_train_vectorized = vectorizer.transform(X_train)  
X_train_vectorized
```

```
Out[5]: <4179x7466 sparse matrix of type '<class 'numpy.int64'>'  
        with 55283 stored elements in Compressed Sparse Row format>
```

```
In [6]: # Initialize the model and fit with the training data  
NBmodel = MultinomialNB()  
NBmodel.fit(X_train_vectorized, Y_train)
```

```
Out[6]: ▾ MultinomialNB  
        MultinomialNB()
```

```
In [7]: # Make predictions onto the training and testing sets.  
trainPredictions = NBmodel.predict(vectorizer.transform(X_train))  
testPredictions = NBmodel.predict(vectorizer.transform(X_test))
```

Naive Bayes in Python

```
In [8]: # How does the model work on the training set?
confusion_matrix(Y_train, trainPredictions)
```

```
Out[8]: array([[3615,    9],
               [  16,  539]])
```

```
In [9]: # Display that in terms of correct proportions
confusion_matrix(Y_train, trainPredictions, normalize='true')
```

```
Out[9]: array([[0.99751656, 0.00248344],
               [0.02882883, 0.97117117]])
```

99.7% of real messages are classified correctly. Just under 3% of spam messages are thought to be real.

```
In [10]: # How does the model work on the test set?
confusion_matrix(Y_test, testPredictions, normalize='true')
```

```
Out[10]: array([[0.99333888, 0.00666112],
               [0.08854167, 0.91145833]])
```

About 8.9% of spam messages are classified as real in the test data and about 0.7% of real messages are classified as spam.

Naive Bayes in Python

```
In [11]: # Predict some phrases. Add your own.
         NBmodel.predict(
             vectorizer.transform(
                 ["Big sale today! Free cash.",
                  "I'll be there in 5"]))
```

```
Out[11]: array(['spam', 'ham'], dtype='<U4')
```

Exercise Try to add 3 sentences below (first, second and third) as follows:

- use only the word *cash* in the first one;
- use only the word *sale* in the second one;
- use both *sale* and *cash* in the third one (try one that is different from "Big sale today! Free cash."

```
In [12]: # Predict new some phrases. Add your own.
         test_new = NBmodel.predict(
             vectorizer.transform(
                 ["Big sale today! Free cash.",
                  "I'll be there in 5",
                  "First sentence here!",
                  "Second sentence here!",
                  "Third sentence here!"]))
```